SPACE RACCOON: Image Processing Payload

Software Requirements Specification

Field	Value
Document ID	SR-FSW-PAY-001
Version	0.1 (DRAFT)
Date	10/5/2026
Author(s)	William Goldman, Ethan Li, Brendan Desrosiers,
Reviewers	[WE NEED PEOPLE AT SOME POINT 🙏]
Status	Draft for team review

---

1. Purpose

This document specifies the flight software changes needed to operate an image processing payload on SPACE RACCOON. The payload is a camera plus an onboard compute module that runs machine learning (ML) inference on captured images. It sends compact results, and selected images, to the mainboard for downlink.

The mainboard flight software (FSW-mainboard) already contains a payload framework inherited from the CMU Argus mission, built for an NVIDIA Jetson running vision-based orbit determination. This specification builds on that framework and does not replace it.

---

2. Scope

2.1 In scope

- Mainboard FSW changes to command, power, monitor, and retrieve data from the payload
- more…

2.2 Out of scope (covered in separate documents)

- anything?

---

3. Background: Existing Argus Payload Framework

The following already exist and should be reused:

`flight/apps/payload/controller.py` — `PayloadController`, a payload lifecycle state machine:

```
IDLE -> WATCHING -> BOOTING -> ACTIVE -> PROCESSING -> FINISHED
  -> DOWNLOAD -> OFF -> SUCCESS / FAIL
```

- It includes timeouts (boot 120 s, command ack 20 s, download 50 s, shutdown 15 s), a time-ordered experiment command list, periodic payload telemetry requests (every 10 s), and time sync (every 30 s).
- **`flight/apps/payload/uart_comms.py`** — UART link to the payload. Uses fixed 609-byte frames encoded with the splat codec.
- **`flight/apps/payload/download_manager.py`** — File transfer from the payload in 40-packet batches, with missing-fragment tracking and retries.
- **`flight/tasks/payload.py`** — The scheduled payload task (1 Hz) that runs the controller state machine.
- **Satellite state `EXPERIMENT`** (`flight/core/states.py`) — The command task enters it when the payload reports `WATCHING` and leaves it on `IDLE` or `FAIL`.
- **`LOW_POWER` state handling** — Already cuts `JETSON_ENABLE` and `JETSON_SD_REQ`.
- **Ground commands** — `EXPERIMENT`, `SIMPLE_EXPERIMENT`, `DATASET_PROCESSING`, `GET_EXPERIMENT_LIST`.
- **EPS power monitoring** — `JETSON` rail threshold of 16000 mW.

> **Gap analysis:** The Argus framework assumes Argus-specific experiment parameters (orbit determination, Argus camera settings). This spec defines what must change for the RACCOON payload.

---

4. System Overview

4.1 Payload hardware *(to be confirmed by the payload team)*

Component	Specification
Camera	[model], [resolution], [interface: CSI / USB]
Compute module	[Jetson Orin Nano / Raspberry Pi CM4 + accelerator / other]
Storage	[onboard SD/eMMC, capacity]
Mainboard link	UART, [baud rate]
Power	Switched by mainboard (`JETSON_ENABLE`), peak [W], average [W]

4.2 Mission use case

Write something here…

4.3 Data flow

1. Ground uplinks an `IMAGING_EXPERIMENT` command with a start time.
2. At the start time (and only in `NOMINAL` or `EXPERIMENT` state), the mainboard powers on the payload.
3. Payload boots, acknowledges, captures N images, runs inference.
4. Payload returns a results summary and selected image products.
5. Mainboard stores products on its SD card (DataHandler) and powers the payload off.
6. Ground requests results and thumbnails during passes.

---

5. Definitions

Term	Definition
Experiment	One scheduled power-on → capture → process → return cycle
Some more here…	

---

6. Functional Requirements

…

---

7. Fault Management

ID	Fault	Response
Something		

---

8. Interfaces

8.1 Some interface 1

here

9. Resource Budgets

Resource	Budget
Power	Payload peak [ ] W, average during experiment [ ] W, experiment energy [ ] Wh, max experiments per orbit [ ]
Downlink	Radio throughput [ ] bytes/pass, results file [ ] bytes, thumbnails [ ] × [ ] bytes. Target: one experiment's results + thumbnails downlinked in ≤ [2] passes
Mainboard SD	Payload allocation [ ] MB
Mainboard RAM	Payload task + DownloadManager must stay within [ ] KB
CPU	Payload task `main_task()` must complete in < [50] ms per cycle

10. Code Changes (planned)

File / Component	Change

11. Verification

Unit tests (`tests/`, run in CI)

- here

Emulator tests

- here

Hardware-in-the-loop

here

---

12. Open Questions

1. How do we do it?

---

13. Milestones

Milestone	Date	Description
M1	[date]	Spec reviewed and approved; ICD drafted
M2	[date]	Emulated payload + unit tests in CI
M3	[date]	Commands, telemetry, and storage implemented in emulator
M4	[date]	Bench integration with real payload hardware
M5	[date]	Full experiment cycle verified; code freeze for flight

---

Revision History

Version	Date	Author	Description
0.1	[date]	[author]	Initial draft